

Семинар по C++ №12

Наследование

Виды наследования:

- public - public и protected наследуются без изменения доступа
- protected - все данные становятся protected
- private - все данные становятся private

	Исходный модификатор доступа		
	public	private	protected
public-наследование	public	private	protected
private-наследование	private	private	private
protected-наследование	protected	private	protected

Layout



N.B. Empty Base Optimization

Если базовый класс пустой, то его размер равен 1, но при наследовании он не занимает дополнительного места

Parent constructor

- Порядок вызова конструкторов:
 1. Поля родителя
 2. Тело конструктора родителя
 3. Поля наследника
 4. Тело конструктора наследника
- При уничтожении деструкторы вызываются в обратном порядке
- В конструкторе наследника можно явно вызвать конструктор родителя, но не проинициализировать поля

using

- Можно наследовать конструкторы родителя с помощью `using` - сгенерируется конструктор для наследника, вызывающий конструктор Base, поля конструкторов одинаковые
- Конструктор копирования не наследуется, потому что тогда можно было бы не скопировать поля наследника
- Также можно использовать `using` с методами и полями, чтобы избежать перекрытия методов родителя методами наследника

Приведение типов при наследовании

- Можно принимать наследника по ссылке/указателя на родителя
- Если принимать по значению, то происходит slicing:
Будет сгенерирован конструктор `Base(Derived d)`,
который забирает поля `Base` у `d` себе
- В обратную сторону не работает
- Если у `Base` есть нетривиальный конструктор копирования, то вызовется он

Множественное наследование